

AGENTS.md: SANKALP prototype baseline (v2, zero-backend)

Read this whole file first. Work **one phase at a time**, then stop and report. Do not start the next phase unless asked.

0. Decision: no runtime backend

SANKALP is a privacy-first PWA for the SANGYAN Investor Resilience Hackathon, Track D. The deck promises "100% in-browser", "no servers", "near-zero running cost". A server adds risk, cost and a weaker privacy story, and nothing in the prototype needs one at runtime.

So "making it fully functional" means:

1. Replace fake parts of the frontend with real logic (prices, P&L, ML model, weekly report, CSV import).
2. Use **build-time scripts on the dev machine** (model training, sample data, Bhashini audio) that output **static files** into `public/`.
3. Deploy as **static hosting** (GitHub Pages, Netlify or Cloudflare Pages). The app works offline.

An optional live Bhashini proxy is a stretch goal in section 8. Do not build it unless asked.

1. Guardrails (a violation fails the project)

1. **No user data leaves the browser.** No endpoint, analytics, or third-party call ever receives trades, P&L, reasons, PIN or typed/spoken text.
2. **Rules override ML.** `evaluate()` in `src/engine.js` sets `lock` from the rule score only (`score >= 2`). A model may only set `nudge`. It never locks or unlocks.
3. **No advice.** No buy/sell/hold, targets, predictions, "you will lose". Wording stays "possible pattern". Prices are labelled "simulated, not predictions".
4. **Never a permanent block.** Every lock ends after the timer and can be exited.
5. **No monetisation, ads, tracking.**
6. **Offline first.** Everything works with the network off after first load.
7. **Every library, dataset and API is listed in README.md** under "Third-party disclosure".

2. Keep it light (rules for you, the agent)

- Node ≥ 20 . **No new runtime dependencies. No new dev dependencies** unless I approve. Use `node:test`, `node:assert`, global `fetch`, `node:fs`, `node:crypto`.
- Plain JavaScript ESM, same style as the existing code. No TypeScript, no bundler changes, no Docker, no database, no Python.
- Build scripts in `scripts/` run on the dev machine only; the app never imports them.

- Verify with `npm test` and `npm run build`. Do not open the browser agent or take screenshots unless I ask. Do not leave watchers or dev servers running.
- Small diffs. Do not reformat files. Do not rename existing exports.
- Budgets: total `dist/` (JS + CSS + model) < 400 kB gzip excluding audio; model file <= 10 KB; `evaluate()` < 5 ms; first load works on a slow 3G profile.

3. Current frontend facts (verified; do not break)

```
src/engine.js      exports T, setLocked, isLocked, submitOrder, evaluate,
replay, PRESETS
src/vault.js       exports EMPTY, unlock, save, wipe (localStorage keys sk_salt,
sk_log; PBKDF2 150k -> AES-GCM 256)
src/i18n.js        exports SRC, WHY, HZ, RS, L, t (en/hi/mr, helper
R(en,hi,mr))
src/App.jsx        state, placeOrder, endLock, loadRameshDemo, handleCsvImport,
fetch/XHR counter `reqN`
src/components/    GateScreen, Header, Navbar, PracticeTab, JournalTab,
InsightsTab, TrustTab, LockModal, Toggle
public/sw.js       service worker (cache name 'sankalp-v1', stale-while-
revalidate for all GET)
src/engine.test.mjs run by `npm test`
vite.config.js     base './' (keep relative paths so it works on any static
host or subfolder)
```

Vault shape today: `{h:[{t,size,lev,pnl,src}], j:[{t,inst,side,why,hz,lock,note}], m:{locks, cancel, proceed, jl, ms?, fl?}, g:{name,amt}|null}`. `evaluate(o,h,cap,now,th)` returns `{lock, reasons:[{k,v}], score, nudge, z}`.

Backlog (gaps found in the code)

ID	Where	Problem
B1	<code>App.jsx</code> ~L92	Prices are <code>Math.random()</code> walks, different on every load.
B2	<code>App.jsx</code> ~L150	Order P&L is <code>Math.random()</code> . Not honest, not reproducible.
B3	<code>engine.js</code>	"ML" is only a z-score. The deck promises an int8 model.
B4	<code>InsightsTab</code>	Report is session totals, not weekly. Lock events have no timestamps.
B5	<code>handleCsvImport</code>	Naive <code>split(',')</code> , no validation, no feedback, ignores <code>side</code> . No sample files.
B6	<code>TrustTab</code>	Bhashini toggle is hard-disabled. No Bhashini audio exists. Voice is browser Web Speech only.
B7	<code>App.jsx</code> <code>reqN</code>	Counts every fetch/XHR with no meaning. Needs categories.
B8	<code>sw.js</code>	New deploys can stay stale; model and audio are not precached.
B9	thresholds	X/Y hard-coded in <code>engine.js</code> , no versioned, reviewable config file.
B10	deck vs code	Deck says IndexedDB and ONNX/TF.js; code uses localStorage and no model. See section 7.

4. Target layout (additions only)

```
src/market.js          seeded simulated prices, deterministic by (seed,
tickIndex)
src/anomaly.js          int8 model runtime (~40 lines, no deps), lazy-loaded
src/data.js            getStatic(path): the ONLY place that calls fetch;
counts requests by category
src/migrate.js          vault v1 -> v2
public/config/thresholds.json  versioned presets (+ "reviewedBy": null)
public/data/ramesh.csv  calm.csv  nightowl.csv  generated by script, header:
time,side,size,lev,pnl
public/model/anomaly.int8.json      generated by script
public/tts/{en,hi,mr}/{phraseId}.wav  generated by script (optional,
needs Bhashini keys)
scripts/gen-synthetic.mjs          seeded synthetic sessions (+ sample CSVs)
scripts/train-anomaly.mjs          tiny MLP in pure Node -> int8 JSON
scripts/guardrail-check.mjs        fails on advice-like phrases in i18n strings
scripts/warm-tts.mjs              dev-only, reads .env, calls Bhashini, writes wav
files
docs/model-card.md
.env.example                     BHASHINI_USER_ID= , BHASHINI_ULCA_API_KEY= (dev
machine only; .env git-ignored)
```

5. Commands

bash

```
npm install          # once
npm run dev          # vite on :5173
npm test             # engine + market + anomaly + migration + csv + guardrail c
npm run build        # -> dist/
npm run preview      # serve dist/ locally
npm run gen          # node scripts/gen-synthetic.mjs      (CSVs + synthetic set)
npm run train        # node scripts/train-anomaly.mjs
npm run warm-tts     # node scripts/warm-tts.mjs           (optional, needs keys)
```

Extend `test` to: `node src/engine.test.mjs && node --test tests/ && node scripts/guardrail-check.mjs`. Put new tests in `tests/*.test.mjs`.

6. Phases (each ends green)

Phase 0: plumbing and categories (B7, B8, B9)

- ☐ Add `(src/data.js)` `(getStatic(path))` with 1 s timeout; allow-list `(config/thresholds.json)`, `(data/*.csv)`, `(model/anomaly.int8.json)`, `(tts/**)`. Counts: `(own)` (same-origin static), `(external)` (must stay 0), `(unexpected)` (must stay 0). Keep the existing `(fetch)`/XHR wrapper in `(App.jsx)` feeding the same counters.
- ☐ TrustTab shows "Requests carrying your data: 0" and a small "Static files loaded: N". Update `(L.zero)` in `en/hi/mr`.
- ☐ `(public/sw.js)`: bump cache name, precache the app shell plus the three static folders on install, delete old caches on activate, keep offline fallback.
- ☐ `(public/config/thresholds.json)` + boot-time merge in `(App.jsx)` over `(PRESETS)`, with range clamping (`(streak)` 2..6, `(maxLev)` 2..20, `(cool)` 10..600); ignore on failure.
Done when: `(npm test)` and `(npm run build)` pass.

Phase 1: honest prices and P&L (B1, B2)

- ☐ `(src/market.js)`: mulberry32 PRNG; `(priceAt(inst, tickIndex, seed))`; `(tickIndex = floor(Date.now()/1500))`; bases A=100, B=250, C=50; ~0.5% volatility. Deterministic, no `(setInterval)` state drift, same feel as today. Seed in `(public/config/thresholds.json)` or constant.
- ☐ Orders record `(entryPx)`. P&L is marked to market after a fixed hold (default 5 ticks): `(pn1 = round(size*lev*(pxExit/pxEntry-1)*(side==='buy'?1:-1)))`. Push to history on close. `(submitOrder)` and the lock check stay the only entry path. Keep the demo behaviour of `(loadRameshDemo)`.
- ☐ No `(Math.random())` left in `(src/App.jsx)` order or price logic.
- ☐ Tests: determinism (same seed + tick = same price), P&L sign for buy/sell.
Done when: tests pass and the Ramesh demo still locks.

Phase 2: sample data and CSV import (B5)

- ☐ `(scripts/gen-synthetic.mjs)` (seeded) writes `(public/data/{ramesh,calm,nightowl}.csv)` and `(.cache/synthetic.json)` (git-ignored). `(ramesh)` = 3 losses then escalation; `(calm)` = no locks; `(nightowl)` = late-night bursts.
- ☐ Harden `(handleCsvImport)`: quoted fields, header check, skip bad rows, cap 5,000 rows, honour `(side)`, show "imported N, skipped M" in `en/hi/mr`. Move parsing to `(src/csv.js)` so it is testable.

- ❑ TrustTab: "Try a sample log" buttons (consent toggle still applies to file import; samples are static files).
- ❑ Tests: parser edge cases; importing `(ramesh)` then evaluating the escalation order gives `(lock:true); (calm)` gives no lock.

Phase 3: real on-device anomaly model (B3)

- ❑ Features (8): `(size/avgSize)`, `(lev/20)`, `(lastPnl/cap)`, `(lossStreak/5)`, `(tradesIn30min/5)`, `(sin(hour))`, `(cos(hour))`, `(riskySource)`.
- ❑ `(scripts/train-anomaly.mjs)`: MLP 8 -> 8 -> 1 (ReLU, sigmoid), pure Node, fixed seed, trained on the synthetic set (behaviours: calm, fomo, revenge, night). Export int8 weights + per-tensor scale + feature mean/std to `(public/model/anomaly.int8.json)`.
- ❑ `(src/anomaly.js)`: lazy `(loadModel())` and `(score(features))`. In `(evaluate())` use it **only** to set `(nudge)` (`(z>2)` OR `(p>=0.8)`). If the model fails to load, fall back to z-score.
- ❑ `(docs/model-card.md)`: features, synthetic data, held-out precision/recall/false-lock rate, limits. Say plainly that validation on real behavioural data is future work (the deck asks mentors for such data).
- ❑ Tests: ML never changes `(lock)`; p95 inference < 5 ms over 1,000 calls; file <= 10 KB; `(npm run train)` twice gives identical output.

Phase 4: Bhashini audio, build-time only (B6)

- ❑ `(scripts/warm-tts.mjs)` (dev only): read `(.env)`, call Bhashini (config call, then compute call with a `(tts)` task) for the fixed phrase set (`(lockM)`, `(breathe)`, `(nudge)`, `(poss)`) in hi and mr, write `(public/tts/{lang}/{phraseId}.wav)`. Follow <https://bhashini.gitbook.io/bhashini-apis>; verify endpoints and headers there before coding. Skip politely if keys are missing.
- ❑ `(LockModal)`/`(App.speak())`: if the Bhashini toggle is on and a static wav exists, play it with `(Audio)`; otherwise use `(speechSynthesis)` as today. Enable the toggle (remove `(dis)`) with text: "Spoken lock message by Bhashini (pre-recorded; nothing is sent)".
- ❑ Voice input stays browser Web Speech (opt-in). Bhashini speech-to-text is **not** used (it would send audio). Whisper-tiny/Vosk is stretch only.
- ❑ Add 2 more languages later with a build-time `(translate-phrases)` script. Not in this phase.
Done when: app plays the static audio offline; with no wav files it still works.

Phase 5: weekly calm-down report (B4)

- ❑ Vault v2 in `(src/migrate.js)`: `(v:2)` and `(d.lk=[{t, rules:[keys], action:'save'|'skip'|'cancel'|'fp'}])`. Silent migration on load, keep all existing fields. Record `(lk)` in `(placeOrder)`/`(endLock)`.

- ☐ `InsightsTab`: last 7 days (pauses, reflection %, late-night orders, cancelled orders), a per-day mini list, plus the existing replay lab. "Patterns only. No advice." in en/hi/mr. Do not claim "most users cancel"; show only the user's own numbers.
- ☐ Tests: migrating a v1 object loses nothing; report numbers change after a lock/cancel cycle.

Phase 6: compliance, static deploy, demo

- ☐ `scripts/guardrail-check.mjs`: scan all `src/i18n.js` strings for `buy now`, `sell now`, `target`, `guaranteed`, `will lose`, `will gain`, `sure shot`, and bare `tip` (allow-list the existing option "Saw a tip"). Fail on hit.
- ☐ README: "Third-party disclosure" (React, Vite, Bhashini TTS pre-generated audio, browser Web Speech API, any datasets), "What leaves your device: nothing", a "Limits" section (see section 7), update stack summary and run instructions.
- ☐ Deploy: `npm ci && npm run build`, publish `dist/` to GitHub Pages / Netlify / Cloudflare Pages. Confirm it works under a subfolder (relative base) and offline after first load.
- ☐ Demo script file `docs/demo.md` (the 6-step flow already in README).

7. Honest limits (put in README "Limits"; do not hide)

- **B10 deck mismatches.** The code uses encrypted `localStorage`, not IndexedDB, and a tiny int8 JS model, not ONNX/TF.js. Cheapest fix: reword the deck ("encrypted local storage", "tiny int8 model in the browser"). Migrating the vault to IndexedDB is a stretch.
- **PIN strength.** A 4-digit PIN has 10,000 combinations; PBKDF2 slows guessing but this is a privacy deterrent, not strong encryption. Say so; allow longer PINs.
- **Web Speech API** sends audio to the browser vendor (e.g. Google in Chrome). It is opt-in and off by default. Truly local speech needs Whisper-tiny/Vosk (stretch).
- **ML** is trained on synthetic data and only nudges. Thresholds are provisional until reviewed by mentors (`reviewedBy` in the config).
- **Replay Lab** is a scripted simulation, not a pilot result.

8. Stretch (only if asked)

Live Bhashini proxy as a Cloudflare Worker (keys server-side, fixed `phraseId` only, no free text); Whisper-tiny lazy-loaded local STT; ONNX export; IndexedDB vault; opt-in false-lock counters for threshold tuning; browser extension to gate broker sites.

9. Definition of done

- ☐ Ramesh demo: lock, 60 s timer, mr/hi/en message, reason saved encrypted, unlock.
- ☐ Prices and P&L deterministic; no `Math.random()` in order or price logic.
- ☐ Model loads, < 5 ms, only nudges.

- ☐ Weekly report from vault v2; v1 vaults migrate.
- ☐ Trust tab: data-carrying requests 0, external 0, unexpected 0.
- ☐ Works offline; static deploy works; `npm test` green; README disclosure and limits written.

10. Reporting

After each phase reply with: files changed, commands run and results, what you skipped and why. Short. If a requirement here conflicts with section 1, section 1 wins; tell me.